MapForAll

Project structure explained

A plain-language guide to the Map_me repository. Read this if you are new to the project and need to understand what the product does, who uses it, and which folders matter.

What it is

A map + chat assistant for discovering real places in Kigali (and later World Cup host cities). People talk in French, English, or Kinyarwanda; the app answers with real map pins, not invented addresses.

Why two big folders

client/ is the website users see. agents/ is the AI brain that searches places and builds answers. Everything else supports deploy, docs, or hardware.

Product nickname

In code and docs you will also see "Hodari" (Swahili for capable / brave). MapForAll is the product name on the landing page; Hodari is the agent system behind it.

Tip: start with pages 2–3, then skim the folder map on page 4.

1. What MapForAll does

Imagine someone lands in Kigali and types: "I want tomatoes near me" or "un café accessible à Kimihurura". MapForAll finds real shops and cafés, puts them on a map, and can suggest a short route.

The three kinds of users

1) Visitors / customers — open the map, chat, save places, get directions. 2) Business owners — claim a shop, upload photos and hours, see official views and attendance. 3) Admins / moderators — approve or reject listings, manage users and reports.

What "grounded" means here

The assistant must not invent places. Results come from the project database (owner-claimed and validated listings) and from Google Maps when needed. Only validated, non-paused places appear in public search.

Languages

- English, French, and Kinyarwanda UI strings live in client/locales/.
- Search understands product words across languages (tomatoes / tomates / inyanya).

Out of scope for the MVP (so you do not look for them in every folder)

- Full booking and payments as a marketplace (billing exists for plans, not place tickets).
- Heavy vector search for every city (Phase 2 idea; Kigali uses geo + text search first).

2. A request through the system

Follow one user sentence from the browser to the map pins. This is the mental model you need before opening files.

Step A — Browser (client/)

The chat page sends the message (and optional GPS) to a Next.js API route. The UI also owns the map, place cards, and voice button.

Step B — API bridge

client/app/api/chat talks to the Python agent server. The website does not contain the big AI logic; it streams the answer back.

Step C — Agents (agents/hodari)

The Orchestrator routes the ask. For “find places”, discovery tools extract intent (e.g. market / tomatoes), query MongoDB near the user, then optionally Google Maps.

Step D — Data

MongoDB Atlas stores users, places, reviews, views, and interactions. Agents and the Next.js API reach it through a MongoDB MCP HTTP sidecar in local/dev.

Step E — Back to the map

Candidates stream to the client. Pins appear, the place sheet can open, and a view is counted only for published places (not for owners or admins).

3. The repository at a glance

Think of the repo as a small company: shopfront, brain, shipping, manuals, and gadgets.

client/
The website and its own APIs. Next.js App Router, React, Tailwind. This is what you open in the browser.

agents/
The AI runtime (Hodari). Python + Google ADK. Searches places, ranks them, drafts the reply.

deploy/
Cloud Run YAML and scripts to publish agent services.

infra/
Docker / Cloud Run examples, including the MongoDB MCP sidecar image.

docs/
Architecture, billing, security, and roadmaps. Start with Hodari_System_Architecture.md.

hardware/ + firmware/
ESP32 RFID sketches for shop attendance (optional physical device).

scripts/
One-off utilities at the repo root (dedupe, verification helpers).

4. Inside client/ — the website

If you change what people see, you are almost always under client/.

Important subfolders

- app/ — pages and API routes. Examples: /chat, /login, /business/dashboard, /admin/places.
- components/ — reusable UI. MapView is the map; ChatPanel is the conversation; admin/ and business/ hold console screens.
- lib/ — shared TypeScript logic (sessions, places, moderation, view counting, i18n helpers).
- locales/ — en.json, fr.json, rw.json translation dictionaries.
- scripts/ — maintenance: seed demo data, reindex search fields, reconcile view counters.

Pages by audience

- Public: landing, login, chat/map, saved places, public place link /p/[id].
- Owner: /business/onboarding and /business/dashboard (stats, team, listing status).
- Staff: /admin/* for places, users, reports, moderation, audit, settings.

Why API routes live next to the UI

Next.js route handlers in app/api/* keep secrets on the server (session cookies, Mongo MCP, Stripe). The browser never talks to MongoDB or Vertex AI directly.

5. Inside agents/ — the AI brain

Hodari is a multi-agent system, but for the MVP it usually runs as one process with in-process routing.

The four roles (keep this number)

- Orchestrator — entry point; loads profile/history; streams the final answer.
- Planner — turns a free-text ask into a structured Plan (goal + constraints).
- Explorer / Map — finds places (database + Maps) and returns a CandidateSet.
- Itinerary — orders stops and computes routes when a multi-stop plan is needed.

Where to look in code

- hodari/agent.py — wires the agents together.
- hodari/sub_agents/ — planner, explorer, itinerary modules.
- hodari/tools/discovery.py — place search ranking for chat ("tomatoes near me").
- hodari/tools/query_intent.py — maps words like tomatoes/tomates to a market search.
- hodari/tools/mongo_tools.py & places_tools.py — database and vector helpers.
- tests/ — pytest; run these after changing search or ranking.

Rule of thumb

UI bug → client/. Wrong places in chat, bad ranking, or odd agent text → agents/. Missing place after an admin approve → check validation/search indexing on the client API side and discovery filters.

6. Data, deploy, and hardware

The remaining folders exist so the product can store truth, ship, and optionally scan staff badges.

MongoDB collections (conceptually)

users (accounts & roles), places (listings with status and location), interactions (likes/visits for personalization), plus reviews, view events / daily stats, attendance, reports, and admin audit logs.

Listing status (single source of truth)

A place moves through draft → submitted / in_review → validated (public) or changes_requested / rejected. paused and archived hide it. Soft delete keeps a restore window. Chat search only returns validated, not paused places.

deploy/ and infra/

YAML for Cloud Run agent services and Docker examples. Locally you typically run: Mongo MCP on :3100, ADK agents on :8000, Next.js on :3001 (or :3000 if free).

hardware/ and firmware/

ESP32 RFID reader code for business attendance. Useful for owners tracking who is on site; not required to try the chat map.

7. How to explore the code as a beginner

A practical order of reading so you are not lost in hundreds of files.

- Read README.md and docs/Hodari_System_Architecture.md once.
- Open client/app/page.tsx and client/app/chat/page.tsx to see the public UX entry.
- Open client/components/MapView.tsx and ChatPanel.tsx — the two main surfaces.
- Open client/app/api/chat/route.ts — the bridge to the agents.
- Open agents/hodari/agent.py then tools/discovery.py — how places are found.
- Open client/app/admin/places/page.tsx — how staff publish a listing.
- Skim client/locales/en.json — product vocabulary in one place.

Local run (high level)

- Start MongoDB MCP with your Atlas URI on port 3100.
- In agents/: create the venv, install requirements, run the ADK api_server for hodari.
- In client/: npm install, copy .env.example → .env, npm run dev.
- Never commit secrets (.env, API keys). Demo passwords only belong in seed scripts.

Glossary

ADK = Google Agent Development Kit. MCP = Model Context Protocol (tool server). SSE = Server-Sent Events (streaming chat). Validated = approved for the public map. Local business = informal / neighbourhood shop flag with ranking bonus.

8. Compact folder tree

A short tree for orientation. Build caches (node_modules, .venv, .next) are omitted on purpose.

```
Map_me/
├── client/                ← website (Next.js)
│   ├── app/               pages + API routes
│   ├── components/        UI building blocks
│   ├── lib/               shared server/client logic
│   ├── locales/           en · fr · rw
│   └── scripts/           seed / reindex / maintenance
├── agents/                ← AI runtime (Python ADK)
│   ├── hodari/            orchestrator + tools
│   ├── services/          Cloud Run entrypoints
│   ├── scripts/           seed places, embeddings
│   └── tests/             pytest
├── deploy/cloud_run/      publish agents
├── infra/                 MCP + Cloud Run examples
├── docs/                  architecture & roadmaps
├── hardware/ esp32-rfid/  attendance reader
├── firmware/ dumpInfo/    MCU helper sketch
└── scripts/               repo utilities
```